Authentication in Node.js using Express, MongoDB, Sessions, Bcrypt, and Twig

Overview

This tutorial explains how to implement user authentication in a Node.js application using the following stack:

- **Express.js** for routing
- **MongoDB + Mongoose** for storing user data
- **Bcrypt** for password hashing
- **Express-session** for session management
- **Twig** as the view engine

The tutorial covers:

1. Understanding the authentication flow
2. Password hashing and comparison using bcrypt
3. Implementing user registration
4. Implementing user login
5. Creating protected routes
6. Handling logout and restricting access
7. Using middleware to enforce authentication

---

1. Authentication Workflow

1.1 What Authentication Means

Authentication verifies whether a user is allowed to access certain pages by validating login credentials (username + password).

1.2 Core Process

1. User submits username & password
2. Express server receives the credentials
3. Server forwards credentials to the database
4. Database checks if they match existing records
5. If the user exists, the server creates a session
6. Client is redirected to a protected page
7. Only authenticated users can access protected routes

---

2. Required Packages

Install all required packages:

```
npm install express mongoose express-session bcryptjs twig
```

Include them in your project:

```
import express from "express";
import session from "express-session";
import bcrypt from "bcryptjs";
import mongoose from "mongoose";
```

---

3. Setting Up MongoDB and Mongoose

3.1 Connect to MongoDB

```
mongoose.connect("mongodb://localhost:27017/user-crud")
  .then(() => console.log("Connected"))
  .catch(err => console.log(err));
```

3.2 User Schema

models/User.model.js:

```
import mongoose from "mongoose";

const UserSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  userpassword: { type: String, required: true }
});

export default mongoose.model("User", UserSchema);
```

---

4. Configuring Express

```
const app = express();

app.use(express.urlencoded({ extended: true }));
app.use(express.json());

app.set("view engine", "twig");
```

5. Implementing Password Hashing (bcrypt)

5.1 Hash Password Before Saving

```
const hashedPassword = await bcrypt.hash(userpassword, 10);
```

10 = salt rounds (recommended)

5.2 Compare Password on Login

```
const isMatch = await bcrypt.compare(userpassword, user.userpassword);
```

6. User Registration

6.1 Registration Form (Twig)

views/register.twig

```
<form method="POST" action="/register">
  <input type="email" name="username" placeholder="Email" required>
  <input type="password" name="userpassword" placeholder="Password" required>
  <button type="submit">Register</button>
</form>
```

6.2 Registration Route

```
app.post("/register", async (req, res) => {
  const { username, userpassword } = req.body;

  const hashedPassword = await bcrypt.hash(userpassword, 10);

  await User.create({
    username,
    userpassword: hashedPassword
  });

  res.redirect("/login");
});
```

7. User Login

7.1 Login Form (Twig)

views/login.twig

```
<form method="POST" action="/login">
  <input type="email" name="username" placeholder="Email" required>
  <input type="password" name="userpassword" placeholder="Password" required>
  <button type="submit">Login</button>
</form>

{% if error %}
<div class="alert alert-danger mt-3">{{ error }}</div>
{% endif %}
```

7.2 Login Route

```
app.post("/login", async (req, res) => {
  const { username, userpassword } = req.body;

  const user = await User.findOne({ username });

  if (!user) {
    return res.render("login", { error: "User not found" });
  }

  const isMatch = await bcrypt.compare(userpassword, user.userpassword);

  if (!isMatch) {
    return res.render("login", { error: "Invalid password" });
  }

  req.session.username = username;

  res.redirect("/home");
});
```

8. Session Setup

```
app.use(session({
  secret: "secret123",
  resave: false,
```

```
  saveUninitialized: false
}));
```

Session is stored as:

```
req.session.username = username;
```

9. Protecting Routes with Middleware

9.1 Middleware Function

```
function checkLogin(req, res, next) {
  if (req.session.username) {
    return next();
  }
  return res.redirect("/login");
}
```

9.2 Apply Middleware on Protected Pages

```
app.get("/home", checkLogin, (req, res) => {
  res.render("home", { username: req.session.username });
});

app.get("/profile", checkLogin, (req, res) => {
  res.render("profile", { username: req.session.username });
});
```

10. Home & Profile Pages (Twig)

views/home.twig:

```
<h1>Welcome, {{ username }}</h1>
<a href="/logout">Logout</a>
```

views/profile.twig:

```
<h1>Your Profile</h1>
<p>User: {{ username }}</p>
<a href="/logout">Logout</a>
```

---

11. Logout Implementation

```
app.get("/logout", (req, res) => {
  req.session.destroy(() => {
    res.redirect("/login");
  });
});
```

---

12. Prevent Logged-In Users from Opening Login Page

```
app.get("/login", (req, res) => {
  if (req.session.username) {
    return res.redirect("/home");
  }
  res.render("login", { error: null });
});
```

Same condition can be added for `/register`.

---

13. Outcome

With this implementation:

- Users can register with a hashed password
- Users can log in using their credentials
- Sessions persist authentication state
- Non-logged users cannot access protected pages
- Logged-in users cannot access the login page
- Logging out destroys the session

This completes the full authentication cycle in a production-ready manner.
